

TOKYO, JAPAN / DECEMBER 11-13, 2025

Paravirt Scheduling: Framework for better CPU usage

Shrikanth Hegde - IBM
Ilya Leoshkevich - IBM





Agenda

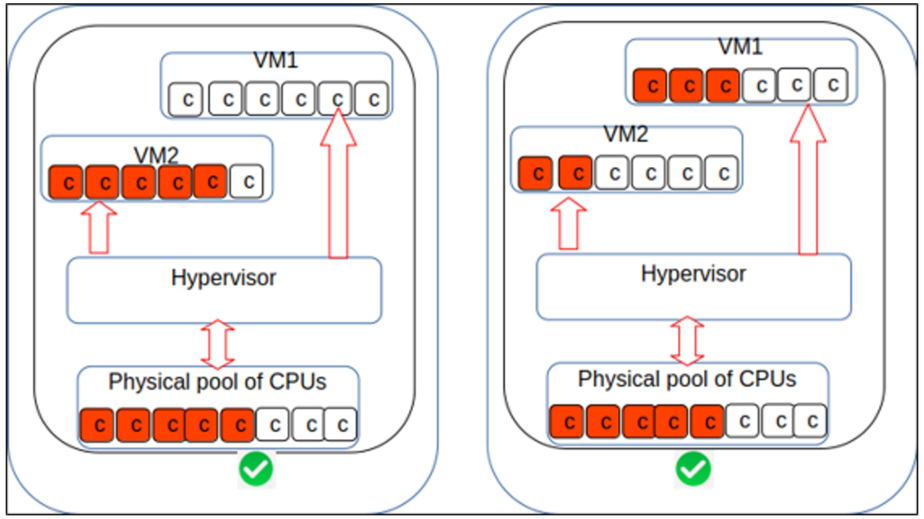
- Problem Statement
- Why Its needed: Why any current method doesn't work.
- Current state: Patches and Approach.
- Implementation details.
- Peformance Numbers
- Open questions

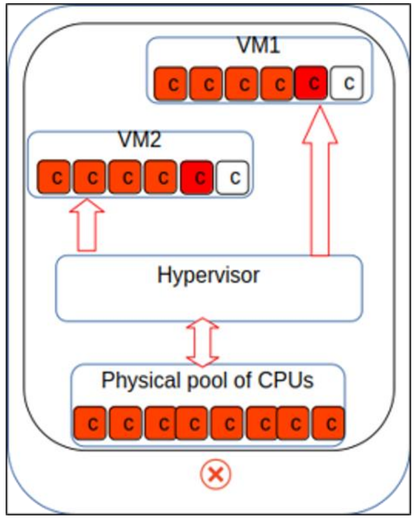


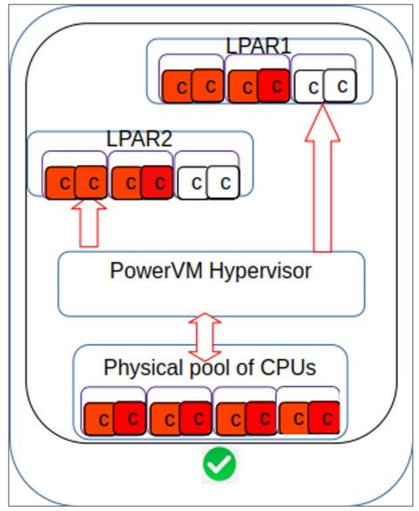
Problem Statement: vCPU Preemption leads to performance impact

Multiple VMs are sharing the underlying CPU resources

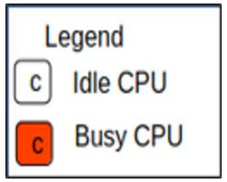
- There is overcommit of CPU resources, i.e., sum of vCPUs > pCPUs
- High CPU utilization in multiple VMs at same time leads to vCPU preemption
- Observed both in enterprise workloads and microbenchmarks
- LPC 2024 talk: https://lpc.events/event/18/contributions/1878/







vCPU – Virtual CPU – CPU in the guest
pCPU – Physical CPU – CPU in the host



Suboptimal placement

Ideal placement

Why it's needed: why no current method works

- **Explicit affinity:** we can't expect a user to set affinity for each workload
- **CPU hotplug:** slow, expensive, breaks user-specified affinities
- **Isolated cpuset partitions:** needs sched domain rebuild, breaks user-specified affinities
- **Capacity aware scheduling:** doesn't push tasks from a CPU under high load

What's needed?

- A lightweight and fast mechanism to move tasks away from specific CPUs
- If task is affined to these CPUs alone, nothing can be done
- Move tasks back quickly when these CPUs are usable again

Current state: Approaches and Patches

New load balancer type

- Cleaner implementation compared to the CPU capacity-based one
- Moves tasks slowly on large systems

[RFC PATCH 1/2] sched/fair: introduce new scheduler group type group_parked

Paravirt Mask and Push current task

- Similar mechanism to CPU offline
- Good performance and Fast
- Simpler implementation compared to load balance stats
- Our proposed variant

[PATCH 00/17] Paravirt CPUs and push task for less vCPU preemption

Implementation details

- Introduce a new cpumask called cpu_paravirt_mask, which contains a list of CPUs marked as "paravirt" (tentative name)
- Make wakeup and load balancing aware of this mask
- Push any running fair/rt task out of these CPUs at tick using stopper thread

powerpc bits

- Get system specific data for the VM, such entitlements
- Periodically compute steal time values
- Adjust number of cores/CPUs to use based on the low/high thresholds

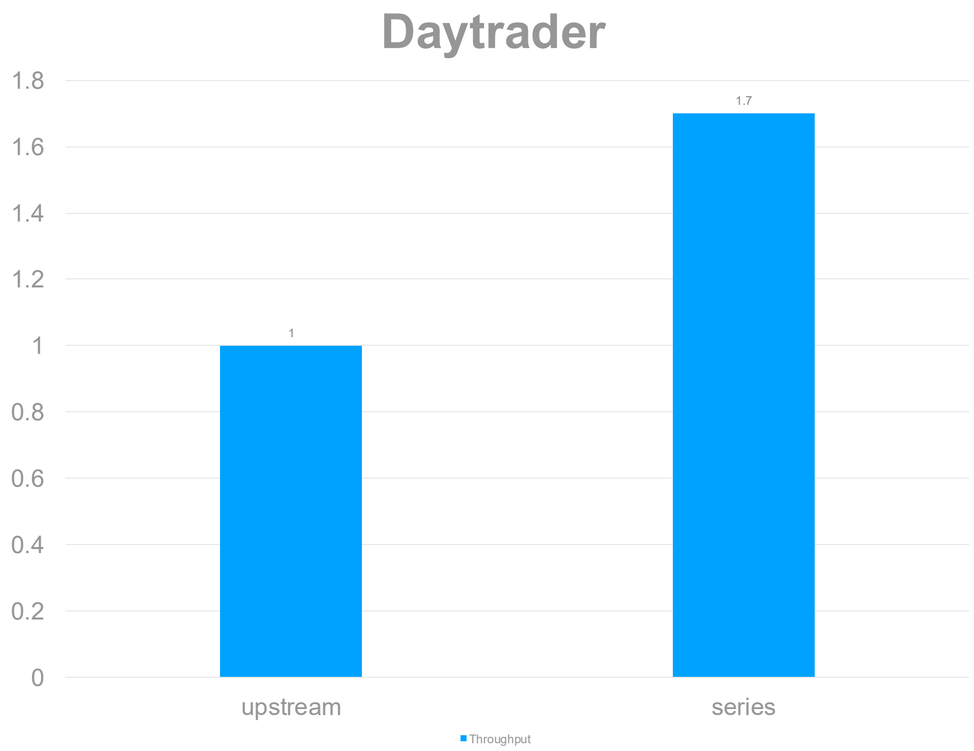
s390 bits

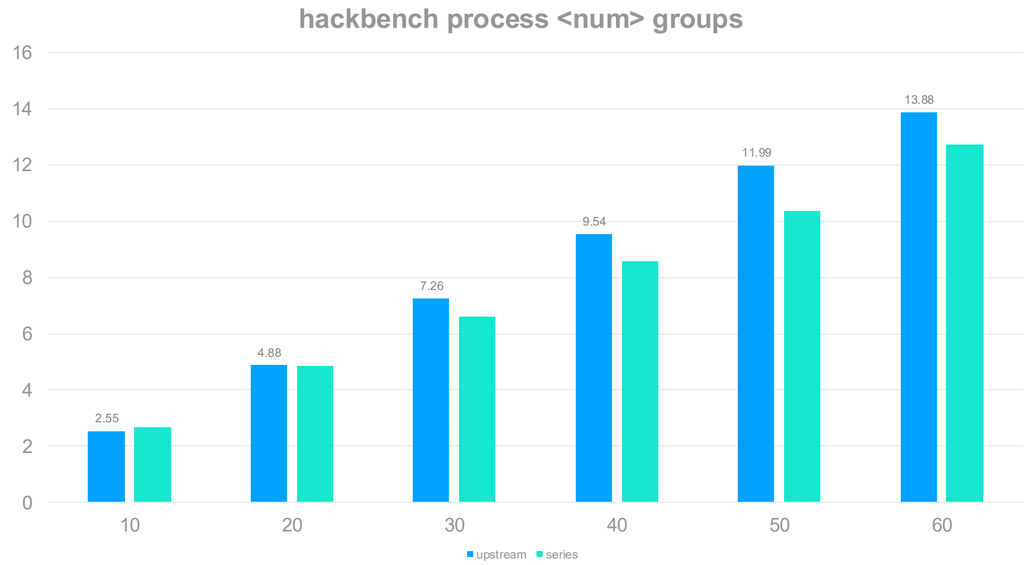
- LPAR: Make Hiperdispatch (existing upstream feature) use cpu_paravirt_mask instead of capacity
- KVM: Add a simple steal time-only heuristic

Performance evaluation

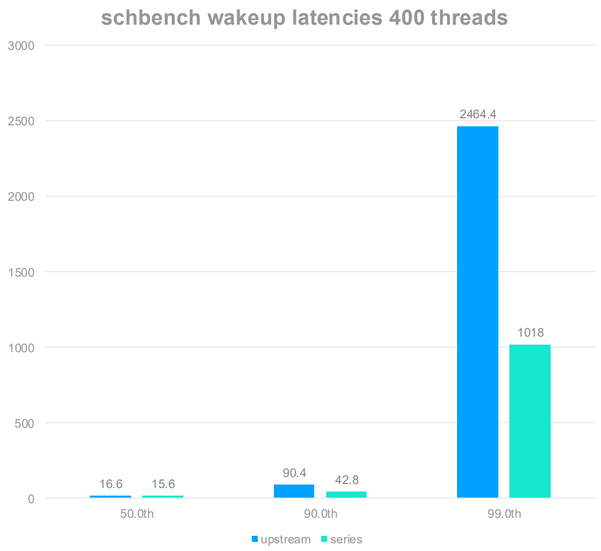
- Build upstream and patched kernels; enable arch-specific heuristics
- Start multiple VMs
- Run the same benchmark inside each VM
- Vary the number of VMs, number of vCPUs, and benchmark parameters
- Measure throughput and latencies

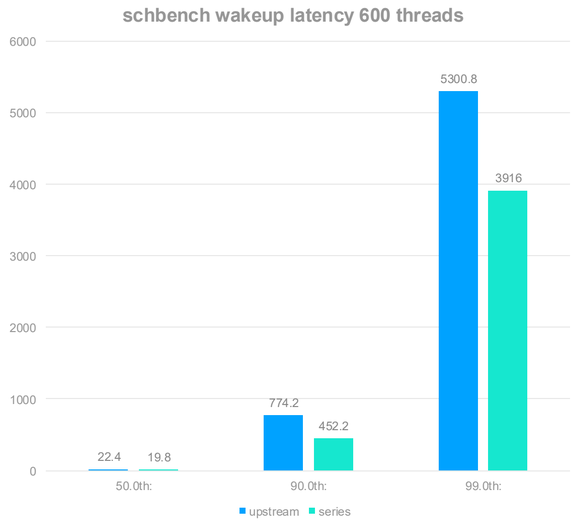
Performance: ppc64le





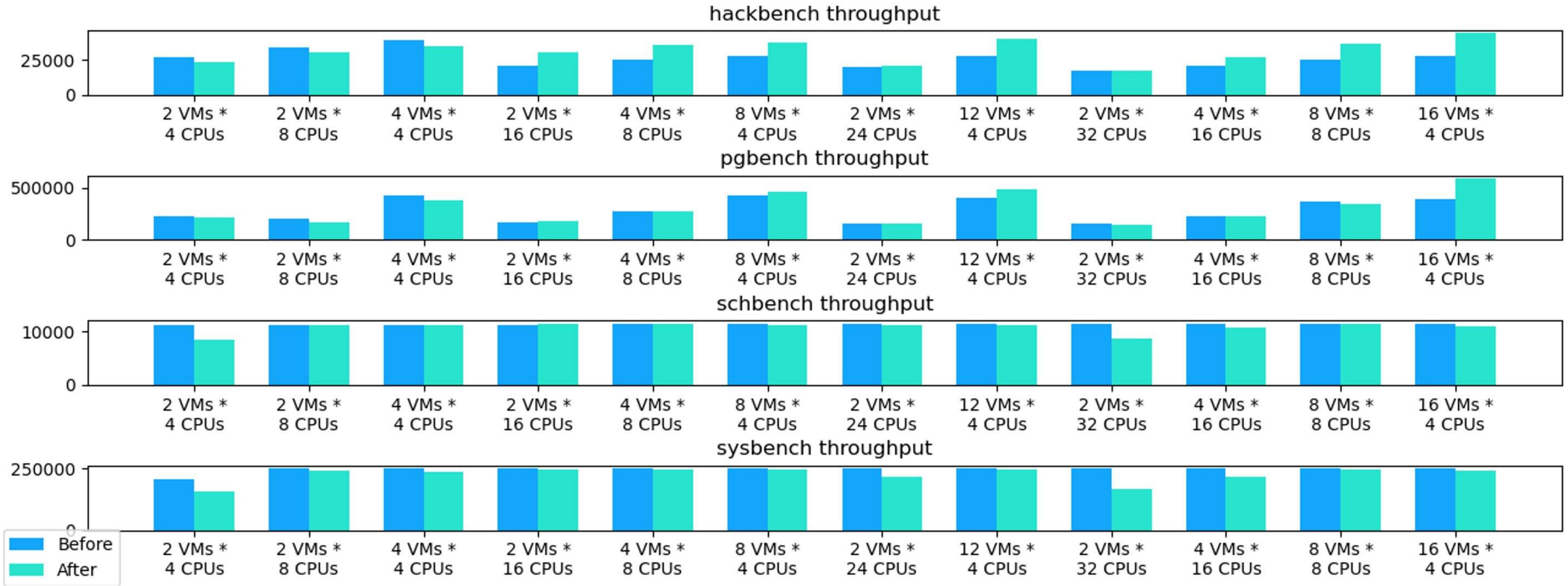
768 CPUs
20-30% steal time in guest
90+% host utilization



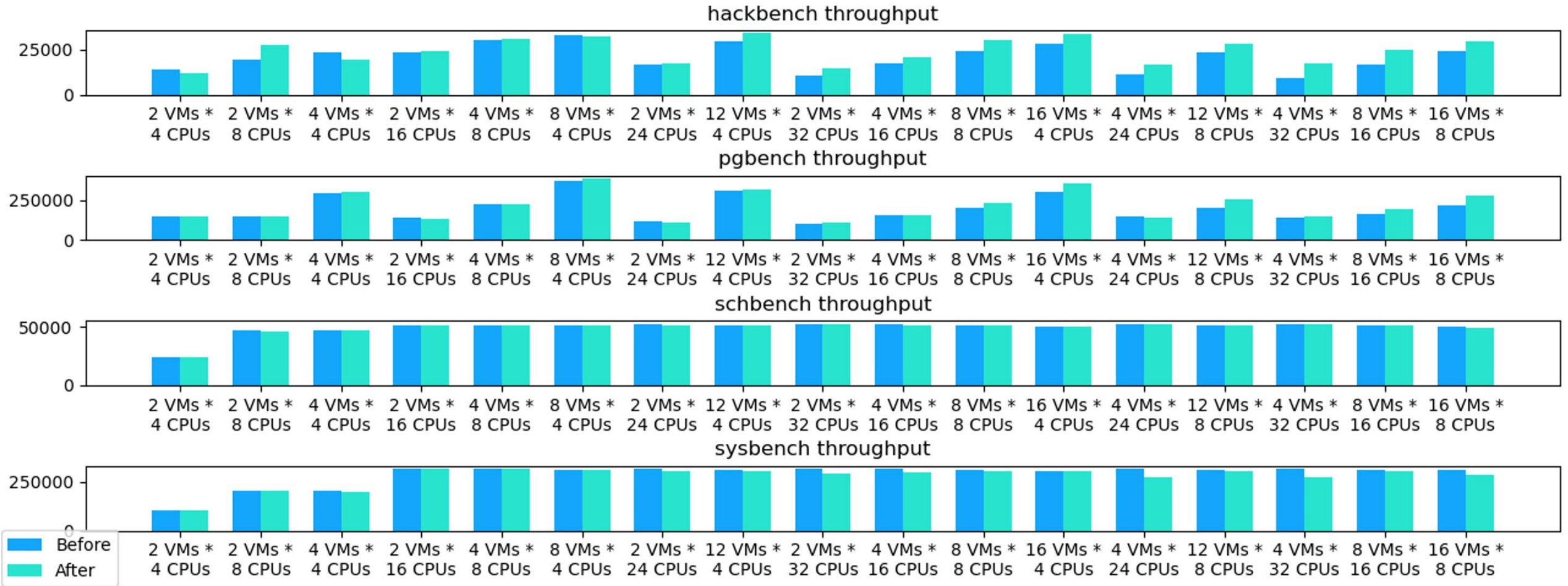




Performance: s390x



Performance: x86_64



東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Open questions

- Is this right approach?

- Naming ("parking" -> "cpu avoid" -> "paravirt")
- Is pushing all tasks at once, as opposed to pushing 1 per tick, worth the additional complexity?
- Generic steal time-based policy
- Augment generic policy with arch-specific hints, like power entitlements or s390 polarization
- Userspace daemons (e.g., irqbalance) should honor this mask

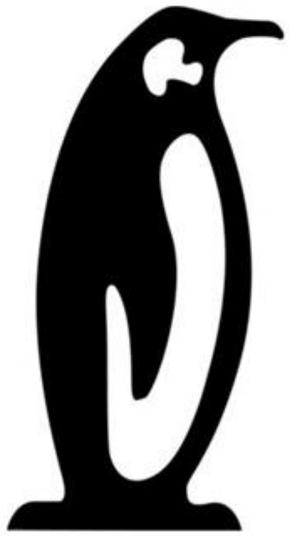
Legal statement

- This work represents the view of the author and does not necessarily represent the view of IBM
- IBM and IBM (logo) are trademarks or registered trademarks of International Business Machines Corporation in the United States and/or other countries
- Linux is a registered trademark of Linus Torvalds
- Other company, product, and service names may be trademarks or service marks of others

Thank you
ありがとう



東京 2025
LINUX
PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

